

Free Claude Code (FCC)

Comprehensive User Guide, Installation, Configuration & Integration Manual

What You Get

- Launch Claude Code with `fcc-claude`, Codex with `fcc-codex`, or Pi with `fcc-pi`.
- Run FCC in the background from a desktop launcher on Windows or macOS.
- Switch among 25 cloud and local providers from the Admin UI.
- Use each coding agent's native model picker.
- Route Fable, Opus, Sonnet, Haiku, and fallback traffic to different models.
- Keep streaming, tool use, reasoning, and image input across compatible models.
- Connect Claude Code and Codex in VS Code or Claude Code through JetBrains ACP.
- Optionally run Claude Code sessions through Discord or Telegram with voice-note transcription.
- Protect the local proxy with optional token authentication.

1. Quick Start

Windows PowerShell Installation

Run the following command in PowerShell:

```
& ([scriptblock]::Create((irm "your_File_path/main/scripts/install.ps1")))
```

Note

Replace `"your_File_path"` with your actual installation path or repository source.

Start FCC

- **Windows:** Open *Free Claude Code* from your desktop or Start menu.
- **macOS:** Open *Free Claude Code* from your desktop or Applications folder.
- **Linux:** Run `fcc-server` in your terminal.

On Windows and macOS, FCC runs in the system tray or menu bar without opening a terminal. Use its menu to open Admin, check server status, restart, or quit. On Windows, left-clicking the tray icon opens Admin directly.

To print the installed Free Claude Code version without starting the server, run:

```
fcc-server --version
```

When using `fcc-server`, keep the terminal open. The Admin UI opens in your browser once the server is healthy by default. Its address is shown in the startup log:

INFO: Admin UI: <http://127.0.0.1:8082/admin> (local-only)

Use the port shown in your terminal if it differs from 8082.

2. Run Your Coding Agent

Start your preferred coding agent using the dedicated CLI launchers:

- **Claude Code:** `fcc-claude`
- **Codex:** `fcc-codex`
- **Pi:** `fcc-pi`

All three launchers use the current Admin UI settings. Use the agent's model picker to choose from the models FCC exposes. Normal CLI arguments still work, for example:

```
fcc-codex exec "hello"
```

`fcc-pi` registers FCC only for that Pi process; your existing Pi settings, sessions, credentials, and extensions remain unchanged.

3. Choose A Provider

1. Open a provider link below for its key, models, or setup instructions.
2. In the Admin UI, open **Model Config** and enter the listed setting.
3. Search the **MODEL** dropdown and select a model. If the provider cannot list models, enter `<provider-id>/<exact-provider-model-id>` manually.
4. Click **Validate**, then **Apply**.

PROVIDER	ADMIN UI SETTING	EXAMPLE MODEL
NVIDIA NIM	NVIDIA_NIM_API_KEY	nvidia_nim/nvidia/nemotron-3-super-120b-a12b
OpenRouter	OPENROUTER_API_KEY	open_router/openrouter/free
Google AI Studio (Gemini)	GEMINI_API_KEY	gemini/models/gemini-3.1-flash-lite
Google Vertex AI	VERTEX_PROJECT_ID + ADC	vertex/google/gemini-3.5-flash
DeepSeek	DEEPSEEK_API_KEY	deepseek/deepseek-chat
Mistral La Plateforme	MISTRAL_API_KEY	mistral/devstral-small-latest
Mistral Codestral	CODESTRAL_API_KEY	mistral_codestral/codestral-latest
OpenCode Zen	OPENCODE_API_KEY	opencode/gpt-5.3-codex
OpenCode Go	OPENCODE_API_KEY	opencode_go/minimax-m2.7

PROVIDER	ADMIN UI SETTING	EXAMPLE MODEL
Vercel AI Gateway	AI_GATEWAY_API_KEY	vercel/openai/gpt-5.5
Amazon Bedrock	AWS_BEARER_TOKEN_BEDROCK	bedrock/openai.gpt-oss-120b
Hugging Face Inference	HUGGINGFACE_API_KEY	huggingface/Qwen/Qwen3-Coder-480B-A35B-Instruct:fastest
Cohere	COHERE_API_KEY	cohere/command-a-plus-05-2026
GitHub Models	GITHUB_MODELS_TOKEN	github_models/openai/gpt-4.1
Wafer	WAFER_API_KEY	wafer/DeepSeek-V4-Pro
Kimi API	KIMI_API_KEY	kimi/kimi-k2.5
Kimi Code	KIMI_CODE_API_KEY	kimi_code/k3
MiniMax	MINIMAX_API_KEY	minimax/MiniMax-M3
Cerebras Inference	CEREBRAS_API_KEY	cerebras/gpt-oss-120b
Groq	GROQ_API_KEY	groq/llama-3.3-70b-versatile
SambaNova	SAMBANOVA_API_KEY	sambanova/Meta-Llama-3.3-70B-Instruct
Fireworks AI	FIREWORKS_API_KEY	fireworks/accounts/fireworks/models/llama-v3p3-70b-instruct
Cloudflare Workers AI	CLOUDFLARE_API_TOKEN + ID	cloudflare/@cf/moonshotai/kimi-k2.6
Z.ai	ZAI_API_KEY	zai/glm-5.2
Ollama Cloud	OLLAMA_API_KEY	ollama_cloud/qwen3-coder:480b
LM Studio	LM_STUDIO_BASE_URL	lmstudio/<model-id>
llama.cpp	LLAMACPP_BASE_URL	llamacpp/<model-id>
Ollama	OLLAMA_BASE_URL	ollama/<model-tag>

Important Provider Notes

- **Mistral Codestral** uses a separate key from Mistral La Plateforme.
- **Kimi Code** subscription keys use `kimi_code/` ; Kimi API credit keys use `kimi/` . Kimi Code plans are for personal interactive coding-agent use under Kimi's community guidelines.
- **OpenCode Zen and OpenCode Go** share `OPENCODE_API_KEY` but use different model prefixes.
- **Amazon Bedrock** uses its Mantle OpenAI-compatible endpoint. Set `BEDROCK_BASE_URL` to the endpoint for the same region as the API key and select one of the models returned by FCC's model picker.

- **Vertex AI** uses Google Application Default Credentials instead of an API key. Locally, run `gcloud auth application-default login` once; service-account files and attached service accounts also work. Set `VERTEX_PROJECT_ID`, and optionally change `VERTEX_LOCATION` from its global default. FCC refreshes expiring access tokens automatically.
- **Cloudflare** requires both its API token and account ID.
- **Ollama Cloud** connects directly to `ollama.com`; use the exact model IDs shown by FCC's model picker. Local Ollama remains available through the separate `ollama/` prefix.
- Prefer tool-capable models for coding agents. Local models also need enough context for the agent's system prompt and tool definitions.

4. Optional Model-Tier Routing

`MODEL` is the fallback for every request. Select a model for `MODEL_FABLE`, `MODEL_OPUS`, `MODEL_SONNET`, or `MODEL_HAIKU` to override an individual Claude Code tier; select *None* to use `MODEL`.

Example: Route Opus to `nvidia_nim/moonshotai/kimi-k2.6`, Sonnet to `open_router/openrouter/free`, Haiku to `lmstudio/qwen3.5-coder`, and keep `MODEL` on `zai/glm-5.2`.

5. Reasoning Control

Open **Admin UI** → **Model Config** → **Reasoning** and select the behavior you want:

SELECTION	BEHAVIOR
From client (default)	Use the effort sent by Claude Code, Codex, or Pi. If none is sent, keep the provider default.
Off	Request reasoning to be disabled.
Low, Medium, High, X-High, or Max	Override the client with the selected reasoning level.
Inherit (Fable, Opus, Sonnet, & Haiku only)	Use the root Reasoning selection.

Providers that do not support a selected control retain their own behavior.

6. Connect Your Client

For terminal use, start `fcc-server`, then run `fcc-claude`, `fcc-codex`, or `fcc-pi`. Use the integrations below for editor setups:

- Claude Code in VS Code
- Codex in VS Code
- Claude Code in JetBrains ACP
- Claude Code still asks you to log in (handled via proxy integration)

7. Optional Integrations

Configure integrations from **Admin UI** → **Messaging**, then click **Validate** and **Apply**.

- Discord bot
- Telegram bot
- Voice-note transcription support

Messaging Commands

USAGE	BEHAVIOR
<code>/stats</code>	Show session state.
Standalone <code>/stop</code>	Cancel all work.
Reply with <code>/stop</code>	Cancel only the selected request while other queued requests continue.
Standalone <code>/clear</code>	Reset all FCC state and remove every tracked message in that chat, including user prompts, voice notes, FCC replies, Telegram's online notice, and the clear command itself.
Reply with <code>/clear</code>	Delete the selected message and its literal platform reply subtree while preserving its ancestors and siblings.